Proyecto Semestral — Algoritmos II

Título: Sistema de Wave Picking para la preparación de pedidos en un almacén (warehouse)

Equipos: 4 estudiantes

---

1. Objetivos Generales

- Desarrollar una aplicación para simular y gestionar la preparación de pedidos dentro de un Warehouse.
- Modelar la estructura física de un Warehouse usando una estructura de datos vista en clases.
- Implementar algoritmos eficientes para determinar recorridos dentro del Warehouse.
- Desarrollar un sistema de Wave Picking que permita agrupar y procesar pedidos de manera eficiente.
- Implementar mecanismos para determinar recorridos entre las diferentes ubicaciones que deben visitar los operarios.
- Analizar y comparar diferentes algoritmos de resolución en términos de costo temporal y espacial.
- Desarrollar una aplicación visual e interactiva que permita observar el funcionamiento de los algoritmos implementados.

2. Introducción

Una empresa de comercio electrónico cuenta con un centro de distribución encargado de almacenar productos y preparar los pedidos realizados por sus clientes.

El Warehouse está compuesto por diferentes pasillos y ubicaciones. Los productos se encuentran distribuidos dentro de dichas ubicaciones.

Durante la jornada se reciben numerosos pedidos. Cada pedido contiene uno o más productos y, por lo tanto, requiere que un operario (*picker*) recorra diferentes sectores del Warehouse para recolectarlos.

Procesar los pedidos de manera individual puede producir recorridos innecesarios. Por este motivo, la empresa utiliza una estrategia denominada **Wave Picking**, mediante la cual varios pedidos son agrupados en una misma *wave* para ser preparados conjuntamente.

Por ejemplo, si dos pedidos contienen productos ubicados en sectores cercanos del Warehouse, puede resultar conveniente procesarlos dentro de la misma wave.

El sistema deberá permitir representar el Warehouse, cargar pedidos y productos, generar waves y determinar los recorridos necesarios para completar cada una de ellas.

El objetivo del proyecto **no** será encontrar matemáticamente la solución óptima global del problema. En su lugar, los estudiantes deberán diseñar estrategias basadas en algoritmos y estructuras de datos estudiados en la materia y de manera individual.

3. Modelo del Warehouse

El Warehouse puede verse como una entidad por sí misma, por lo que deberá representarse mediante una estructura de datos.

3.1. Sistema de coordenadas

El Warehouse se representa sobre un **eje cartesiano** como un gran rectángulo cuya esquina inferior izquierda es la coordenada `(0,0)` y cuya esquina superior derecha es `(ancho_x, largo_y)`. Todos los componentes del Warehouse (racks, puertas, zonas de despacho, pasillos, productos y pickers) se ubican dentro de esas dimensiones.

El espacio se discretiza en **celdas unitarias**. La celda `(x,y)` es el cuadrado cuya esquina inferior izquierda es `(x,y)`, con $0 \leq x < ancho_x$ y $0 \leq y < largo_y$.

Todo elemento rectangular del Warehouse se define por:

Atributo	Significado
`ubicacion`	Coordenada `(x,y)` de la **esquina inferior izquierda**.
`ancho`	Cantidad de celdas que ocupa sobre el eje `x`.

`alto`	Cantidad de celdas que ocupa sobre el eje `y`.

Un rectángulo con `ubicacion = (x,y)` ocupa las celdas desde `(x, y)` hasta `(x + ancho - 1, y + alto - 1)` inclusive. Ningún elemento puede exceder las dimensiones del Warehouse y dos elementos no pueden solaparse.

Dentro del Warehouse encontramos los siguientes elementos:

3.2. Racks

Los racks son las estanterías donde están guardados los productos. Cada rack es un **rectángulo** definido por su `ubicacion (x,y)`, su `ancho` y su `alto`.

Los elementos dentro de un rack solo pueden accederse **desde el costado**, es decir, desde una celda de pasillo adyacente al rack. Por este motivo, uno de los dos lados del rack (`ancho` o `alto`) debe tener **valor 2**: así el rack queda formado por dos "caras" de celdas, y cada celda del rack tiene una cara expuesta a un pasillo, de modo que pueden colocarse elementos en cada uno de los costados.

Por ejemplo, un rack con `ubicacion = (3,5)`, `ancho = 2` y `alto = 6` ocupa las celdas `(3..4, 5..10)`. Las celdas de la columna `x = 3` se acceden desde el pasillo ubicado en `x = 2`, y las de la columna `x = 4` desde el pasillo ubicado en `x = 5`.

Dentro de cada celda `(x,y)` de un rack los productos se ubican a distintas alturas `z` (ver sección 4).

3.3. Puertas (muelles)

Puntos de acceso que conectan el Warehouse con el exterior y permiten la recepción (*inbound*) y/o expedición (*outbound*) de mercadería.

- Las puertas se ubican siempre **sobre el borde** del rectángulo del Warehouse y pueden estar repartidas en cualquiera de los cuatro bordes. Es decir, la celda `(x,y)` de una puerta cumple `x = 0`, `x = ancho_x - 1`, `y = 0` o `y = largo_y - 1`.
- Cada puerta tiene un `tipo`: `INBOUND`, `OUTBOUND` o `AMBOS`.
- Frente a cada puerta hay siempre una zona de despacho asociada (ver 3.4).

3.4. Zonas de despacho

Áreas donde se preparan, consolidan y organizan los pedidos antes de ser cargados y enviados.

- Cada zona de despacho es un **rectángulo** definido por su `ubicacion (x,y)`, su `ancho` y su `alto`.

- Cada zona de despacho está asociada a una puerta (`puerta_id`) y se ubica **frente a ella**: al menos una celda de la zona es adyacente a la celda de la puerta.
- Cada zona tiene un **límite de peso** (`peso_max_kg`): la suma del peso de los items depositados en la zona en un momento dado no puede superar ese límite.
- La zona de despacho es el punto de partida y de llegada del recorrido de un picker (ver sección 8).

3.5. Pasillos

Los pasillos son los espacios que se crean entre los racks, y entre los racks y las zonas de despacho. **Toda celda del Warehouse que no esté ocupada por un rack, una puerta o una zona de despacho es una celda de pasillo**, por la cual los pickers pueden desplazarse.

Los pasillos tienen **dirección (sentido de circulación)**, y la movilidad de los pickers queda definida por esa dirección:

- Un pasillo se declara como un rectángulo (`ubicacion`, `ancho`, `alto`) con un atributo `sentido`:

`sentido`	**Circulación permitida**
`N`	Solo hacia `+y`.
`S`	Solo hacia `−y`.
`E`	Solo hacia `+x`.
`O`	Solo hacia `−x`.
`AMBOS`	Libre, en las cuatro direcciones.

- El sentido restringe el avance **a lo largo** del pasillo. En un pasillo de sentido `N` o `S` el picker puede desplazarse lateralmente (sobre `x`) entre celdas del mismo pasillo para acercarse a un rack, pero nunca avanzar en contra del sentido. Análogamente para `E` y `O` sobre `y`.
- Las celdas de pasillo que no pertenezcan a ningún pasillo declarado se consideran de sentido `AMBOS`.
- Los pasillos declarados no pueden solaparse entre sí ni con racks, puertas o zonas de despacho.

La red de pasillos **no se entrega como un grafo**: a partir de la grilla de celdas y de los sentidos declarados, el equipo deberá construir la estructura de datos que represente la conectividad del Warehouse y sobre la cual se ejecutarán los algoritmos de recorrido.

3.6. Pickers

Los pickers son los operarios que recorren el Warehouse para recolectar los productos de una wave. Cada picker posee:

- Identificador.
- Posición inicial `(x,y)`, que debe ser una celda de pasillo.
- **Capacidad de carga máxima** (`capacidad_carga_kg`): el picker no puede cargar más items que su peso máximo permitido. En ningún momento la suma del peso de los items que transporta puede superar ese valor.

Reglas de movimiento:

- Un picker se desplaza de una celda de pasillo a otra celda de pasillo **ortogonalmente adyacente** (no se permiten movimientos en diagonal). Cada movimiento tiene costo 1 (distancia unitaria).
- Los pickers se mueven exclusivamente por celdas de pasillo, respetando el sentido de circulación de cada pasillo. No pueden atravesar racks, puertas ni zonas de despacho.
- Un picker accede a una celda de rack (para recoger un producto) o a una zona de despacho (para depositar los productos) cuando se encuentra en una celda de pasillo ortogonalmente adyacente a ella.
- Un picker **no puede pasar por encima de otro**: dos pickers nunca pueden ocupar la misma celda en el mismo instante, ni intercambiar sus celdas en un mismo paso (cruzarse). Si dos pickers utilizan el mismo pasillo no pueden chocar: uno deberá esperar o tomar un camino alternativo.

4. Productos

Cada producto deberá poseer como mínimo:

- Identificador único.
- Nombre.
- Ubicación dentro del Warehouse.
- Cantidad disponible.
- Peso.

Ejemplo:

Producto	**Ubicación**	**Stock**
P001	`<x,y,z>`	20
P002	`<x,y,z>`	15

P003	<x,y,z>	30
P004	<x,y,z>	10
P005	<x,y,z>	25

Los productos pueden estar repartidos por todo el Warehouse.

Reglas de ubicación de los productos (items):

- Inicialmente todo producto se encuentra **dentro de algún rack**. Su ubicación es una terna `(x,y,z)` donde `(x,y)` es una celda perteneciente a un rack y `z ≥ 1` es la altura (nivel) a la que está dentro del rack.
- Puede haber items **uno encima de otro**: distintos productos pueden compartir la misma celda `(x,y)` en distintas alturas `z`.
- Cada posición `(x,y,z)` aloja un único producto; `cantidad_disponible` indica el stock en esa posición.
- Cada producto tiene un **peso** (`peso_kg`) por unidad. El peso se utiliza para controlar la capacidad de carga del picker y el límite de peso de la zona de despacho.

5. Pedidos

Cada pedido estará compuesto por uno o más productos.

Cada pedido posee los siguientes valores:

- Identificador.
- Lista de productos.
- Cantidad solicitada de cada producto.
- Prioridad (valor entero entre 1 a 5 donde a mayor valor mayor prioridad).
- Estado.

Ejemplo:

Pedido	**Producto**	**Cantidad**
P01	P001	2
P01	P004	1
P01	P005	3

P02	P002	1
P02	P003	2
P03	P001	1
P03	P003	1
P03	P005	2

Los pedidos podrán encontrarse en alguno de los siguientes estados:

Estado
PENDIENTE
EN_WAVE
EN_PREPARACION
PREPARADO

6. Waves

Una wave representa un conjunto de pedidos que serán preparados conjuntamente. El sistema deberá permitir crear waves a partir de los pedidos pendientes. Cada wave tendrá una capacidad máxima configurable.

Ejemplo (capacidad máxima de una wave: 3 pedidos):

Wave	Pedidos
Wave 1	P01, P02, P03
Wave 2	P04, P05

Una wave deberá respetar las restricciones establecidas por el sistema. Como mínimo deberá considerarse:

- Cantidad máxima de pedidos.
- Cantidad máxima de productos.
- Prioridad de los pedidos.
- Peso total de los productos, que no puede superar la capacidad de carga del picker que preparará la wave.

Los pedidos con mayor prioridad deberán ser considerados antes que los pedidos de menor prioridad.

7. Generación de Waves

Los estudiantes deberán implementar un mecanismo para generar waves a partir del conjunto de pedidos pendientes. No será necesario encontrar la partición óptima de los pedidos. En su lugar, deberá implementarse una estrategia algorítmica **determinista, claramente definida y justificada**.

La estrategia deberá considerar, como mínimo:

- Capacidad de la wave.
- Prioridad de los pedidos.
- Ubicación de los productos.
- Distancia entre las ubicaciones requeridas por los pedidos.
- Peso de los productos y capacidad de carga del picker.

Por ejemplo, una estrategia podría comenzar seleccionando el pedido de mayor prioridad y posteriormente incorporar pedidos que requieran visitar ubicaciones cercanas. Los estudiantes podrán proponer y comparar diferentes estrategias de generación de waves. La estrategia finalmente seleccionada deberá estar justificada desde el punto de vista algorítmico.

8. Preparación de una Wave

Una vez creada una wave, el sistema deberá determinar el recorrido necesario para preparar todos sus pedidos.

Supongamos:

- Zona de despacho: **A**
- Wave 1:

Pedido	**Producto → Ubicación**

P01	Producto → D
P01	Producto → G
P02	Producto → E
P02	Producto → H

El sistema deberá determinar un recorrido que permita visitar las ubicaciones necesarias:

```
A → D → E → G → H → A
```

El recorrido deberá realizarse utilizando **exclusivamente los pasillos existentes** en el Warehouse, respetando el sentido de circulación de cada uno. El recorrido comienza y termina en una celda de pasillo adyacente a la zona de despacho, y cada producto se recoge desde una celda de pasillo adyacente a la celda del rack donde se encuentra (ver 3.6).

9. Estrategias de Recorrido

Cada equipo deberá implementar **al menos dos estrategias diferentes** para determinar el recorrido de una wave.

Por ejemplo:

Estrategia A — Orden de los pedidos

Las ubicaciones se visitan siguiendo el orden en que aparecen los pedidos.

Estrategia B — Cercanía

Luego de visitar una ubicación, se selecciona como siguiente destino una ubicación aún no visitada que se encuentre a menor distancia.

Los equipos podrán proponer otras estrategias siempre que puedan justificarlas algorítmicamente.

Para cada estrategia deberán medir:

- distancia total recorrida;
- cantidad de nodos visitados;
- cantidad de aristas recorridas;
- tiempo de ejecución.

10. Comparación de Algoritmos

El sistema deberá permitir comparar las estrategias implementadas. Para una misma wave deberá ser posible ejecutar diferentes algoritmos y observar sus resultados.

Por ejemplo:

Métrica	Estrategia A	Estrategia B
Distancia total	245	198
Puntos visitados	31	27
Tiempo	4 ms	7 ms

Los estudiantes deberán analizar las diferencias obtenidas. La comparación deberá realizarse utilizando diferentes tamaños y configuración de Warehouse y diferentes cantidades de pedidos.

11. Aplicación Visual

Uno de los componentes fundamentales del proyecto será la creación de una aplicación visual e interactiva. La aplicación deberá permitir visualizar el Warehouse. Para ello deberán poder visualizarse:

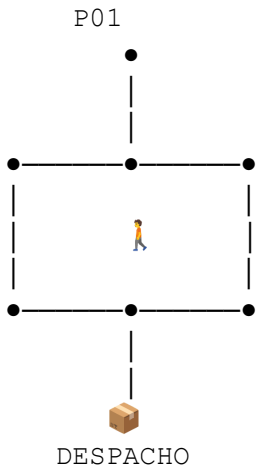
- Pasillos y su sentido de circulación.
- Intersecciones.
- Racks y ubicación de productos.
- Puertas.
- Zona de despacho.
- Pedidos.
- Waves.
- Pickers y recorrido del picker.

12. Simulación Interactiva

La aplicación deberá permitir seleccionar una wave y ejecutar su preparación.

Durante la ejecución deberá poder observarse visualmente cómo el picker (encargado de recoger los productos) se desplaza por el Warehouse.

Por ejemplo:



La aplicación deberá permitir:

- iniciar la simulación;
- pausar la simulación;
- continuar la simulación;
- reiniciar la simulación;
- avanzar paso a paso.

Durante la ejecución deberán mostrarse los productos que van siendo recogidos.

La simulación deberá contemplar la presencia de **más de un picker** desplazándose simultáneamente. En cada paso deberán respetarse las reglas de la sección 3.6: dos pickers no pueden ocupar la misma celda ni cruzarse dentro de un pasillo, y deberá poder observarse cómo se resuelve el conflicto (espera o camino alternativo).

La aplicación deberá permitir observar el funcionamiento de los algoritmos propuestos. La visualización deberá permitir **comprender cómo trabaja el algoritmo**, no solamente mostrar el resultado final.

13. Configuración de Escenarios

La aplicación deberá permitir crear o cargar diferentes escenarios de configuración de Warehouse.

Los estudiantes deberán utilizar estos escenarios para analizar el comportamiento de sus algoritmos.

16. Generación de Escenarios

La aplicación deberá permitir generar Warehouses de manera automática.

La generación deberá permitir configurar, como mínimo:

- cantidad de estantes;
- cantidad de pasillos;
- cantidad de productos;
- localización de los productos;
- cantidad de pedidos;
- cantidad máxima de productos por pedido;
- capacidad de las waves.

El sistema deberá garantizar que el Warehouse generado sea **válido**. Como mínimo:

- todos los elementos están dentro de las dimensiones del Warehouse y no se solapan;
- todos los racks tienen un lado de valor 2;
- todas las puertas están sobre el borde;
- frente a cada puerta hay una zona de despacho;
- todos los productos están ubicados dentro de un rack;
- las ubicaciones requeridas pueden ser **alcanzadas desde la zona de despacho** respetando el sentido de circulación de los pasillos.

17. Persistencia

La configuración del Warehouse, los productos y los pedidos deberá poder guardarse y recuperarse posteriormente.

La aplicación deberá permitir el siguiente flujo:

```
Crear escenario
        ↓
Guardar escenario
        ↓
Cerrar aplicación
        ↓
Cargar escenario
        ↓
Continuar simulación
```

No deberá ser necesario reconstruir manualmente el escenario cada vez que se inicia la aplicación.

18. Requerimientos Técnicos

El proyecto deberá desarrollarse utilizando **Python 3**.

Se deberá implementar una arquitectura modular que permita separar como mínimo:

```
Warehouse
    ↓
Products / Orders
    ↓
Wave Manager
    ↓
Pathfinding
    ↓
Simulation
    ↓
Visualization
```

- La lógica de los algoritmos deberá estar **separada de la interfaz gráfica**.
- Las estructuras de datos principales utilizadas para representar el problema deberán ser **implementadas por el equipo**, salvo aquellas bibliotecas expresamente autorizadas por la cátedra.

19. Restricciones Algorítmicas

El objetivo del proyecto es aplicar los conocimientos de Algoritmos Avanzados.

Por lo tanto, **NO está permitido** resolver el problema utilizando:

- programación lineal;
- programación entera;
- solvers de optimización;
- algoritmos genéticos;
- simulated annealing;
- algoritmos evolutivos;
- otras metaheurísticas.

El proyecto puede resolverse utilizando algoritmos y estructuras de datos estudiados durante la materia.

La utilización de algoritmos adicionales deberá ser previamente discutida en las instancias de revisión del proyecto.

20. Análisis de Complejidad

Para los principales algoritmos implementados deberá analizarse:

- complejidad temporal;
- complejidad espacial;
- mejor caso;
- peor caso;
- comportamiento esperado para diferentes tamaños de entrada.

El análisis deberá estar relacionado con los experimentos realizados.

No será suficiente indicar únicamente la complejidad teórica: deberán presentar **evidencia experimental** del comportamiento de los algoritmos.

21. Experimentación

Cada equipo deberá realizar experimentos utilizando diferentes tamaños de entrada.

Como mínimo deberán analizar:

- tamaño del Warehouse;
- cantidad de pedidos;
- cantidad de productos;
- cantidad de localizaciones;
- cantidad de pasillos.

Para cada escenario deberán registrar:

- tiempo de ejecución;
- distancia recorrida;
- cantidad de elementos procesados;
- cantidad de operaciones relevantes del algoritmo.

Los resultados deberán presentarse mediante **tablas y gráficos**.

22. Preguntas que deberán responder en el informe

El informe final deberá incluir, entre otras, las siguientes preguntas:

1. ¿Cómo fue modelado el Warehouse? (estructura de datos)
2. ¿Qué representación de la estructura de datos utilizaron y por qué?
3. ¿Qué algoritmos utilizaron para resolver los problemas encontrados?

4. ¿Por qué eligieron dichos algoritmos?
5. ¿Qué ocurre si todos los pasillos tienen el mismo largo?
6. ¿Cómo cambia el comportamiento del algoritmo cuando aumenta el tamaño del Warehouse?
7. ¿Cómo determinaron la distancia entre las ubicaciones de una wave?
8. ¿Cómo decidieron qué pedidos pertenecen a una misma wave?
9. ¿Qué estrategia utilizaron para ordenar las ubicaciones de una wave?
10. ¿Qué diferencias existen entre las estrategias implementadas?
11. ¿Cuál presenta menor distancia recorrida?
12. ¿Cuál presenta menor tiempo de ejecución?
13. ¿Existe una estrategia que sea mejor en todos los escenarios?
14. ¿Qué trade-offs encontraron entre tiempo de ejecución y calidad del recorrido?
15. ¿Qué modificaciones realizarían para utilizar el sistema en un Warehouse real?

23. Evaluación del Proyecto

La evaluación tendrá en consideración los siguientes factores:

23.1. Comprensión técnica

- Comprensión completa del código por parte de todos los integrantes.
- Capacidad para explicar las decisiones algorítmicas.
- Capacidad para justificar las estructuras de datos utilizadas.

23.2. Funcionamiento

- Correcta representación del Warehouse.
- Correcta generación y procesamiento de pedidos.
- Correcta generación de waves.
- Correcta ejecución de los algoritmos.
- Correcta simulación de los recorridos.

23.3. Algoritmos y estructuras de datos

- Correcta elección de algoritmos.
- Correcta implementación.
- Correcto análisis de complejidad.
- Eficiencia temporal y espacial.

23.4. Aplicación visual

- Claridad de la representación del Warehouse.
- Visualización de los algoritmos.
- Interactividad.
- Calidad de la simulación.
- Facilidad de uso.

23.5. Experimentación

- Calidad de los escenarios utilizados.
- Cantidad y variedad de experimentos.
- Correcta interpretación de resultados.
- Comparación entre algoritmos.

23.6. Calidad del software

- Arquitectura modular.
- Código claro.
- Documentación.
- Manejo de errores.
- Pruebas.

24. Entrega

Cada equipo deberá entregar:

- Código fuente completo en repositorio GitHub.
- Repositorio Git con el historial de desarrollo.
- README con instrucciones de instalación y ejecución.
- Documentación técnica.
- Informe de análisis algorítmico.
- Resultados experimentales.
- Escenarios de prueba.
- Presentación oral.

25. Demostración Final

Durante la presentación cada equipo deberá realizar una **demostración en vivo**.

Como mínimo deberá:

1. Crear o cargar un Warehouse.
2. Mostrar los productos disponibles.
3. Crear o cargar pedidos.
4. Generar una o más waves.
5. Seleccionar una wave.
6. Ejecutar los algoritmos.
7. Mostrar visualmente los algoritmos.
8. Ejecutar el recorrido del picker.
9. Mostrar los productos recogidos.
10. Comparar al menos dos estrategias.
11. Mostrar las métricas obtenidas.

Finalmente, los integrantes deberán responder preguntas sobre las estructuras de datos, algoritmos, complejidad y decisiones de diseño utilizadas.

26. Consideración Final

El objetivo principal del proyecto **no** es desarrollar un sistema industrial de optimización de Warehouse.

El objetivo es modelar un problema real mediante estructuras de datos y aplicar algoritmos avanzados para resolver sus diferentes componentes, analizar su eficiencia y construir una aplicación que permita visualizar y comprender el funcionamiento de dichos algoritmos.

La calidad de la solución será evaluada principalmente por la correcta aplicación de los conceptos de Algoritmos Avanzados y por la capacidad de los estudiantes para justificar las decisiones tomadas.

Anexos

Anexo A — Ejemplo de escenario (formato JSON)

A.1. Estructura del archivo

Clave	Contenido

`warehouse.dimensiones`	`ancho_x` y `largo_y` del rectángulo del Warehouse, con origen en `(0,0)`.
`warehouse.puertas`	Lista de puertas: `id`, `tipo` (`INBOUND`, `OUTBOUND`, `AMBOS`) y `ubicacion (x,y)` sobre el borde.
`warehouse.zonas_despacho`	Rectángulos (`ubicacion`, `ancho`, `alto`) frente a una puerta (`puerta_id`), con `peso_max_kg`.
`warehouse.racks`	Rectángulos (`ubicacion`, `ancho`, `alto`) con uno de sus lados igual a 2.
`warehouse.pasillos`	Rectángulos (`ubicacion`, `ancho`, `alto`) con `sentido` (`N`, `S`, `E`, `O`, `AMBOS`).
`pickers`	`id`, `posicion_inicial (x,y)` en una celda de pasillo y `capacidad_carga_kg`.
`productos`	`id`, `nombre`, `rack_id`, `ubicacion (x,y,z)` dentro del rack, `cantidad_disponible` y `peso_kg`.
`pedidos`	`id`, `prioridad` (1 a 5), `estado` e `items` (`producto_id`, `cantidad`).
`configuracion`	Parámetros de las waves: `wave_max_pedidos` y `wave_max_productos`.

A.2. Archivo de ejemplo

```
{
  "warehouse": {
    "dimensiones": {
      "ancho_x": 18,
      "largo_y": 16
    },
    "puertas": [
      {
        "id": "PUERTA_OUT_1",
        "tipo": "OUTBOUND",
        "ubicacion": {
          "x": 4,
          "y": 0
```

```
    }
  },
  {
    "id": "PUERTA_IN_1",
    "tipo": "INBOUND",
    "ubicacion": {
      "x": 13,
      "y": 0
    }
  }
],
"zonas_despacho": [
  {
    "id": "ZONA_A",
    "puerta_id": "PUERTA_OUT_1",
    "ubicacion": {
      "x": 2,
      "y": 1
    },
    "ancho": 5,
    "alto": 2,
    "peso_max_kg": 500.0
  },
  {
    "id": "ZONA_B",
    "puerta_id": "PUERTA_IN_1",
    "ubicacion": {
      "x": 11,
      "y": 1
    },
    "ancho": 5,
    "alto": 2,
    "peso_max_kg": 500.0
  }
],
"racks": [
  {
    "id": "R1",
    "ubicacion": {
      "x": 3,
      "y": 5
    },
    "ancho": 2,
    "alto": 6
  },
  {
    "id": "R2",
```

```
    "ubicacion": {
      "x": 8,
      "y": 5
    },
    "ancho": 2,
    "alto": 6
  },
  {
    "id": "R3",
    "ubicacion": {
      "x": 13,
      "y": 5
    },
    "ancho": 2,
    "alto": 6
  }
],
"pasillos": [
  {
    "id": "PAS_V0",
    "ubicacion": {
      "x": 0,
      "y": 5
    },
    "ancho": 3,
    "alto": 6,
    "sentido": "N"
  },
  {
    "id": "PAS_V1",
    "ubicacion": {
      "x": 5,
      "y": 5
    },
    "ancho": 3,
    "alto": 6,
    "sentido": "S"
  },
  {
    "id": "PAS_V2",
    "ubicacion": {
      "x": 10,
      "y": 5
    },
    "ancho": 3,
    "alto": 6,
    "sentido": "N"
```

```
    },
    {
      "id": "PAS_V3",
      "ubicacion": {
        "x": 15,
        "y": 5
      },
      "ancho": 3,
      "alto": 6,
      "sentido": "S"
    },
    {
      "id": "PAS_H_INF",
      "ubicacion": {
        "x": 0,
        "y": 3
      },
      "ancho": 18,
      "alto": 2,
      "sentido": "AMBOS"
    },
    {
      "id": "PAS_H_SUP",
      "ubicacion": {
        "x": 0,
        "y": 11
      },
      "ancho": 18,
      "alto": 5,
      "sentido": "AMBOS"
    }
  ]
},
"pickers": [
  {
    "id": "PK_01",
    "posicion_inicial": {
      "x": 1,
      "y": 3
    },
    "capacidad_carga_kg": 150.0
  },
  {
    "id": "PK_02",
    "posicion_inicial": {
      "x": 16,
      "y": 3
```

```
    },
    "capacidad_carga_kg": 100.0
  }
],
"productos": [
  {
    "id": "P001",
    "nombre": "Monitor 27",
    "rack_id": "R1",
    "ubicacion": {
      "x": 3,
      "y": 5,
      "z": 1
    },
    "cantidad_disponible": 20,
    "peso_kg": 6.5
  },
  {
    "id": "P002",
    "nombre": "Teclado Mecánico",
    "rack_id": "R1",
    "ubicacion": {
      "x": 3,
      "y": 5,
      "z": 2
    },
    "cantidad_disponible": 15,
    "peso_kg": 1.2
  },
  {
    "id": "P003",
    "nombre": "Mouse Inalámbrico",
    "rack_id": "R1",
    "ubicacion": {
      "x": 4,
      "y": 7,
      "z": 1
    },
    "cantidad_disponible": 30,
    "peso_kg": 0.3
  },
  {
    "id": "P004",
    "nombre": "Notebook 14",
    "rack_id": "R2",
    "ubicacion": {
      "x": 8,
```

```
      "y": 6,
      "z": 1
    },
    "cantidad_disponible": 10,
    "peso_kg": 2.1
  },
  {
    "id": "P005",
    "nombre": "Auriculares",
    "rack_id": "R2",
    "ubicacion": {
      "x": 9,
      "y": 9,
      "z": 1
    },
    "cantidad_disponible": 25,
    "peso_kg": 0.4
  },
  {
    "id": "P006",
    "nombre": "Impresora Láser",
    "rack_id": "R3",
    "ubicacion": {
      "x": 13,
      "y": 8,
      "z": 1
    },
    "cantidad_disponible": 8,
    "peso_kg": 9.8
  },
  {
    "id": "P007",
    "nombre": "Cable HDMI",
    "rack_id": "R3",
    "ubicacion": {
      "x": 14,
      "y": 10,
      "z": 1
    },
    "cantidad_disponible": 50,
    "peso_kg": 0.2
  },
  {
    "id": "P008",
    "nombre": "Webcam",
    "rack_id": "R3",
    "ubicacion": {
```

```
      "x": 14,
      "y": 10,
      "z": 2
    },
    "cantidad_disponible": 12,
    "peso_kg": 0.5
  }
],
"pedidos": [
  {
    "id": "PED_01",
    "prioridad": 5,
    "estado": "PENDIENTE",
    "items": [
      {
        "producto_id": "P001",
        "cantidad": 2
      },
      {
        "producto_id": "P002",
        "cantidad": 1
      }
    ]
  },
  {
    "id": "PED_02",
    "prioridad": 3,
    "estado": "PENDIENTE",
    "items": [
      {
        "producto_id": "P003",
        "cantidad": 2
      },
      {
        "producto_id": "P005",
        "cantidad": 1
      }
    ]
  },
  {
    "id": "PED_03",
    "prioridad": 4,
    "estado": "PENDIENTE",
    "items": [
      {
        "producto_id": "P004",
        "cantidad": 1
```

```
        },
        {
          "producto_id": "P006",
          "cantidad": 1
        },
        {
          "producto_id": "P008",
          "cantidad": 3
        }
      ]
    }
  ],
  "configuracion": {
    "wave_max_pedidos": 3,
    "wave_max_productos": 15
  }
}
```

A.3. Mapa del escenario

Representación celda a celda del escenario anterior (el eje `y` crece hacia arriba y el eje `x` hacia la derecha).

```
    0123456789012345678
y=15 ++++++++++++++++++
y=14 ++++++++++++++++++
y=13 ++++++++++++++++++
y=12 ++++++++++++++++++
y=11 ++++++++++++++++++
y=10 ^^^##vvv##^^^##vvv
y= 9 ^^^##vvv##^^^##vvv
y= 8 ^^^##vvv##^^^##vvv
y= 7 ^^^##vvv##^^^##vvv
y= 6 ^^^##vvv##^^^##vvv
y= 5 ^^^##vvv##^^^##vvv
y= 4 ++++++++++++++++++
y= 3 +K++++++++++++++K+
y= 2 ..DDDDD....DDDDD..
y= 1 ..DDDDD....DDDDD..
y= 0 ....P........P....
```

Símbolo	Significado

`#`	Celda de rack.
`P`	Puerta (sobre el borde).
`D`	Zona de despacho.
`K`	Posición inicial de un picker.
`^ v`	Pasillo con sentido `N` / `S`.
`> <`	Pasillo con sentido `E` / `O`.
`+`	Pasillo declarado con sentido `AMBOS`.
`.`	Celda de pasillo no declarada (sentido `AMBOS`).

En este escenario, por ejemplo, el producto `P001` (celda `(3,5)` del rack `R1`, columna izquierda) se recoge desde la celda de pasillo `(2,5)`, que pertenece a `PAS_V0` (sentido `N`). El producto `P003` (celda `(4,7)`, columna derecha de `R1`) se recoge desde la celda `(5,7)`, que pertenece a `PAS_V1` (sentido `S`).